

Hierarchical Communication Optimization for Distributed DNN Training

Om Shivam Verma (B.S. Student)
Indian Institute of Technology Kanpur

Preeti Malakar
Indian Institute of Technology Kanpur

I. INTRODUCTION

Motivation: Large scale deep neural network models use a lot of training data, evident from the large datasets curated for their training, such as ImageNet. Thus it is necessary to use high performance computing (HPC) systems for training of these large-scale models. Various techniques have been developed for parallelizing model training - data-parallelism, model-parallelism, and the more recent FSDP (Fully Sharded Data Parallel).

Limitation of state-of-the-art: As we increase the number of nodes, each technique incurs significant communication overheads. Horovod [4], a data-parallel deep learning training framework, uses global Allreduce calls that lead to communication bottlenecks, as shown in Figure 1. [1] introduces a multi-leader communication scheme for MPI_Allreduce. Similarly, [5] analyzes deep learning on distributed-memory-based GPU-accelerated systems. [3] proposed a scheme for MPI_Alltoall and MPI_Sendrecv with focus on node allocation and topology in HPC systems. We build upon this scheme to mitigate communication bottlenecks.

Key contributions: We implemented a hierarchical intra- and inter-node communication scheme for MPI_Allreduce to improve the overall execution time of Horovod. We achieved this by decreasing the number of processes involved in inter-node communications. The reduction in transfer time is noticeable when the number and communication volume of inter-node transfers are significant. We also evaluate our optimization on various configurations to obtain the best speed-up.

Limitations of the proposed work: This communication scheme may show less improvement for small data and for high-bandwidth systems. Additionally, the number of leaders needs to be tuned for optimal efficiency and is non-trivial as it depends on other factors that are challenging to consider.

II. COMMUNICATION OPTIMIZATION IN HOROVOD

Horovod is a general framework for distributed training of deep learning networks using data-parallelism, which requires each process to have its own copy of the entire model. The training data is distributed across all the processes such that all processes have distinct and balanced data. Following this, each process performs forward and backward propagations simultaneously. After the backward pass, the calculated gradients are averaged over all the processes. This ensures that learning from each process is accumulated globally. MPI_Allreduce is utilized for this operation. Global MPI_Allreduce calls cause major communication bottlenecks on higher process counts as shown in Figure 1. We observe increase in Allreduce time closely following increase in Horovod time.

Our communication scheme requires two sub-communicators from the global communicator. First, the intra-node communicators are based on a user-defined number k , representing the number of leader processes per node. Every leader process is

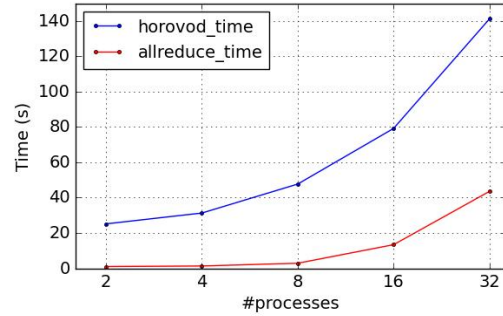


Fig. 1: Horovod scaled upto 32 processes across 2 machines. The training loop runs 30 times with batch size of 32 images.

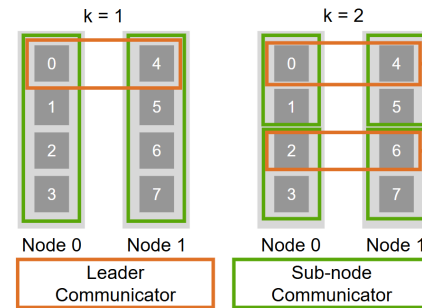


Fig. 2: Number represents the process rank. The hierarchy of communicators for 1 leader (left) and 2 leaders (right) per node is shown. Orange shows *leader communicator*, while green shows *sub-communicators* per node.

associated with a set of ranks within the same node, which together constitute the *sub-node communicator*. Second, all the leader processes from every node together form the *leader communicator*. The scheme has been portrayed in Figure 2. We can tune k for best performance. We use a three-step communication scheme. First, we do an Allreduce or a Reduce in the sub-node communicators, accumulating averages of the associated ranks in the leader processes (both Reduce and Allreduce accomplish this). Second, an Allreduce is called in the leader communicator, distributing the final average to all the leader processes. Finally, each leader rank broadcasts the obtained average to all the ranks in its sub-node communicator.

In our scheme, taking $k = 1$ has the lowest inter-node transfers involved. However, intra-node communications, though shorter in time, are not negligible. Moreover, they include two steps of the process. With more processes per node, intra-node transfers take up more time. Increasing the value of k would reduce the time in intra-node transfers but increase inter-node movements. Thus, a trade-off is involved in choosing k . Further, in the presence of multiple sockets in nodes, choosing k to be the number of sockets so that each has one leader node would reduce inter-socket transfers in addition to inter-node transfers.

III. EVALUATION

We evaluated our implementation using two setups. One is the now-decommissioned HPC2010 supercomputer at Indian Institute of Technology Kanpur for studying the behavior of our hierarchical communication scheme. It consists of Infiniband-based interconnect with Intel Xeon X5570 processor (Nehalem) nodes consisting of 8 cores. We scaled up to 32 nodes with 8 ranks per node, averaging each data point over 5 runs. Results presented in Figure 3 reveal that using Reduce operation at the first intra-node aggregation instead of Allreduce consistently results in 10–30% lower times for more than 32 processes (4 nodes). Considering that this includes only intra-node communications, it is purely dependent on lower number of message transfers. Allreduce needs more communications than Reduce because it requires the results to be sent to all the nodes, in addition to reduction. Step-by-step profiling confirms that only the first step shows these variations, with the remaining steps showing similar times and trends.

The second setup consists of two machines in the Department of Computer Science and Engineering, IIT Kanpur. These are Intel Xeon Gold 5120 (Skylake) nodes with 56 cores each. We scale on up to 32 processes across the two machines averaging each data point over 3 runs to analyze the effects of our optimization on Horovod. The PMPI interface is used to override the MPI_Allreduce calls in Horovod for utilizing our hierarchical implementation. We use synthetic data to run training loops for the DL model ResNet50 [2] with the Stochastic Gradient Descent optimizer, for which the size of parameters is 97.49 MB. In strong scaling, we run 256 training loops. While in weak scaling, each process runs 30 training loops, irrespective of the number of processes. The experiment is run for different number of leaders. The results are shown in Figure 4. Here $k_leader_allreduce$ refers to k leaders per node, and *simple_allreduce* is using the default Allreduce. For 16 and more processes, we observe that the 4-leader communication scheme gives a speed-up of upto 7% compared to *simple_allreduce*. As the number of processes increases, the default Allreduce uses more inter-node data movement. While hierarchical communication reduces inter-node communications, there is an increment in time spent on intra-node communications. However, for a larger number of processes, the reduction in number of inter-node transfers overcomes the loss from intra-node messages.

IV. CONCLUSIONS AND FUTURE WORK

We identified that the MPI_Allreduce, used in data-parallel deep network training is an important bottleneck in deep neural network training. We implemented a three-step leader-based hierarchical communication scheme for optimizing MPI_Allreduce calls by reducing inter-node message transfers. Then, we evaluated the implementation of MPI_Allreduce both standalone and integrated with Horovod. We saw improvements in training time with 4 leaders per node on 32–256 processes.

In future, we aim to scale Horovod on larger systems and for larger workloads. This work may also be extended to GPU-based systems. We observed several other communication bottlenecks in Horovod, we plan to perform a detailed profiling of the same to reveal further prospects for optimization.

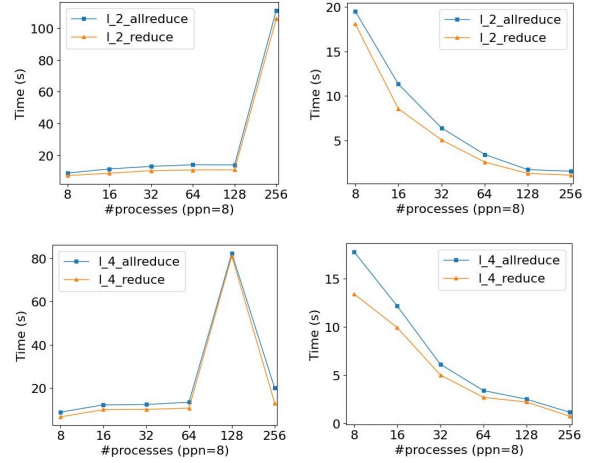


Fig. 3: Weak scaling (left) with 2560 MB data per process and strong scaling (right) with a total of 5120 MB data, distributed over 10 calls of MPI_Allreduce on HPC2010, with each data point averaged over 5 runs.

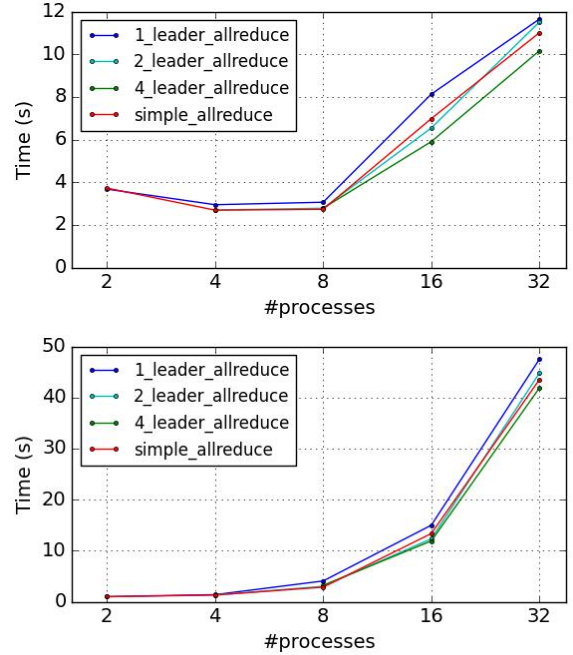


Fig. 4: Strong (top) and weak (bottom) scaling of Allreduce called by Horovod on 2 machines.

REFERENCES

- [1] Mohammadreza Bayatpour, Sourav Chakraborty, Hari Subramoni, Xiaoyi Lu, and Dhabaleswar Panda. Scalable reduction collectives with data partitioning-based multi-leader design. 2017.
- [2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep Residual Learning for Image Recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.
- [3] Mohit Kumar and Preeti Malakar. Hierarchical communication optimization for fft. In *2022 IEEE/ACM International Workshop on Hierarchical Parallelism for Exascale Computing (HiPar)*, pages 12–21, 2022.
- [4] Alexander Sergeev and Mike Del Balso. Horovod: fast and easy distributed deep learning in TensorFlow, 2018.
- [5] Truong Thao Nguyen, Mohamed Wahib, and Ryousei Takano. Hierarchical Distributed-Memory Multi-Leader MPI-Allreduce for Deep Learning Workloads. In *2018 Sixth International Symposium on Computing and Networking Workshops (CANDARW)*, pages 216–222, 2018.